

PPDB Al-Hasra

Virtual Account API

Standar BI SNAP — Autentikasi HMAC-SHA256

Dokumentasi teknis lengkap untuk integrasi API Virtual Account sistem PPDB Al-Hasra. Mencakup spesifikasi semua endpoint, mekanisme autentikasi, format request & response, kode error, dan contoh implementasi dalam berbagai bahasa pemrograman.

GET

/api/va/{va_number}

VA Inquiry by Path Parameter

GET/POST

/api/va/inquiry

VA Inquiry by Request Body

POST

/api/va/payment-notify

VA Payment Notification

POST

/api/btn/callback

BTN Legacy Callback Webhook

DITERBITKAN OLEH

PPDB Al-Hasra — Tim Teknis Integrasi

VERSI & TANGGAL

v1.0.0 — 16 August 2026

Daftar Isi

ISI DOKUMEN

1

Gambaran Umum API

1.1

Deskripsi Sistem & Tujuan

1.2

Daftar Endpoint

1.3

Alur Sistem Virtual Account PPDB

2

Autentikasi & Keamanan (HMAC-SHA256)

2.1

Header Autentikasi yang Wajib

2.2

Cara Membuat Signature (Langkah Demi Langkah)

2.3

Fitur Keamanan Tambahan

2.4

Konfigurasi Environment Server

3

Rate Limiting & Environment

4

Endpoint: VA Inquiry (GET by Path)

4.1

Path Parameter & Headers

4.2

Contoh Request & Response

5

Endpoint: VA Inquiry (GET/POST by Body)

5.1

Request Body / Query String

5.2

Contoh Request & Response

6

Endpoint: VA Payment Notification

6.1

Request Body & Validasi

6.2

Logika Pemrosesan

6.3

Contoh Request & Semua Skenario Response

7

Endpoint: BTN Legacy Callback

7.1

Request Body & Response Format

8

Kode Respons & Error Handling

8.1

Tabel Kode Respons Lengkap

8.2

Format Respons Error Standar

9

Contoh Implementasi Lengkap

9.1

PHP (cURL) — Inquiry & Payment Notify

9.2

JavaScript (Node.js) — Inquiry & Payment Notify

9.3

Python (requests) — Inquiry & Payment Notify

10

Integrasi Postman

10.1

Collection Variables & Pre-request Script

11

FAQ & Troubleshooting

1. Gambaran Umum API

1.1 Deskripsi Sistem & Tujuan

PPDB VA API menyediakan antarmuka standar untuk mengintegrasikan sistem Virtual Account (VA) Bank BTN ke dalam platform PPDB Al-Hasra. API ini mengikuti spesifikasi **Standar Nasional Open API Pembayaran Bank Indonesia (BI SNAP)** dengan autentikasi HMAC-SHA256.

Melalui API ini, sistem bank atau payment gateway dapat melakukan inquiry data tagihan VA siswa dan mengirimkan notifikasi pembayaran secara otomatis dan aman.

BASE URL (DEV)

http://localhost:8000

AUTENTIKASI

HMAC-SHA256 Signature

STANDAR

BI SNAP

VERSI DOKUMEN

v1.0.0 — 2026

FORMAT DATA

JSON (application/json)

RATE LIMIT

60 request / menit

1.2 Daftar Endpoint

METHOD	ENDPOINT	FUNGSI	AUTH	RATE LIMIT
GET	/api/va/{va_number}	VA Inquiry by Path Parameter	HMAC-SHA256	60/menit
GET/POST	/api/va/inquiry	VA Inquiry by Request Body	HMAC-SHA256	60/menit
POST	/api/va/payment-notify	Notifikasi Pembayaran VA	HMAC-SHA256	60/menit
POST	/api/btn/callback	BTN Legacy Callback Webhook	Tidak diperlukan	—

1.3 Alur Sistem Virtual Account PPDB

1. Siswa Daftar

→

2. Generate VA BTN

→

3. Siswa Transfer

→

4. Inquiry / Callback

→

5. Status → Sukses

→

6. Pilih Jadwal Ujian

1. **Siswa Daftar** — Membuat akun dan memilih unit tujuan (SMP/SMA/SMK)

2. **Generate VA** — Menekan tombol “Bayar via VA BTN” untuk mendapatkan nomor VA 17 digit

3. **Transfer** — Siswa mentransfer biaya pendaftaran ke nomor VA tersebut

4. **Verifikasi** — Bank mengirim notifikasi ke `/api/va/payment-notify` atau `/api/btn/callback`, atau admin menekan tombol “Cek Status VA” (Inquiry)

5. **Status Update** — Sistem mengupdate status pembayaran siswa menjadi `success`

6. **Lanjut Proses** — Siswa dapat memilih jadwal ujian dan mengunduh kartu ujian

2. Autentikasi & Keamanan

2.1 Header Autentikasi yang Wajib

Semua endpoint (kecuali `/api/btn/callback`) dilindungi middleware **HMAC-SHA256 Signature Authentication** standar BI SNAP. Setiap request **wajib** menyertakan 3 header berikut:

HEADER	TIPE	KETERANGAN	CONTOH NILAI
X-CLIENT-KEY	string	Identifier klien yang telah disetujui sistem	PPDBALHASRA
X-TIMESTAMP	datetime	Waktu saat ini dalam format ISO 8601. Toleransi ±5 menit dari server.	2026-08-16T10:30:00+07:00
X-SIGNATURE	string	HMAC-SHA256("{CLIENT_KEY} {TIMESTAMP}" , CLIENT_SECRET) → hex lowercase	a3f9d2c1b8e7...

2.2 Cara Membuat Signature (Langkah Demi Langkah)

- 1

Siapkan `CLIENT_KEY` dan `CLIENT_SECRET` — Dikonfigurasi via environment variable `VA_INQUIRY_CLIENT_KEY` dan `VA_INQUIRY_CLIENT_SECRET` di file `.env`.
- 2

Buat timestamp ISO 8601 — Format: `YYYY-MM-DDTHH:mm:ss+07:00`. Pastikan waktu server sudah disinkronkan dengan NTP server (toleransi ±5 menit).
- 3

Buat string yang akan di-sign — Gabungkan `CLIENT_KEY` dan timestamp dengan separator pipe (`|`): `stringToSign = "{CLIENT_KEY}|{TIMESTAMP}"`
- 4

Hitung HMAC-SHA256 — `signature = HMAC-SHA256(stringToSign, CLIENT_SECRET)`, kemudian encode hasilnya sebagai **hex lowercase**.
- 5

Sertakan 3 header dalam setiap request — `X-CLIENT-KEY`, `X-TIMESTAMP`, dan `X-SIGNATURE` wajib ada di setiap request.

JAVASCRIPT — MEMBUAT SIGNATURE (MENGUNAKAN CRYPTOJS)

```
// npm install crypto-js (atau gunakan built-in 'crypto' di Node.js)
const CryptoJS = require('crypto-js');

const clientKey    = 'PPDBALHASRA';
const clientSecret = 'AEkvExg92F';

// Langkah 2: Buat timestamp ISO 8601
const timestamp = new Date().toISOString(); // "2026-08-16T10:30:00.000Z"

// Langkah 3: Buat string-to-sign
const stringToSign = `${clientKey}|${timestamp}`;

// Langkah 4: Hitung HMAC-SHA256
const signature = CryptoJS.HmacSHA256(stringToSign, clientSecret)
    .toString(CryptoJS.enc.Hex); // hasil hex lowercase

// Langkah 5: Sertakan di headers
const headers = {
  'X-CLIENT-KEY' : clientKey,
  'X-TIMESTAMP'  : timestamp,
  'X-SIGNATURE'  : signature,
  'Accept'       : 'application/json',
};
```

PHP — MEMBUAT SIGNATURE (MENGUNAKAN HASH_HMAC BAWAAN PHP)

```
<?php

$clientKey    = 'PPDBALHASRA';
$clientSecret = 'AEkvExg92F';

// Langkah 2: Buat timestamp ISO 8601
$timestamp    = (new DateTime())->format(DateTime::ATOM);
// Contoh: "2026-08-16T10:30:00+07:00"

// Langkah 3: Buat string-to-sign
$stringToSign = $clientKey . '|' . $timestamp;

// Langkah 4: Hitung HMAC-SHA256 (hasil otomatis lowercase hex)
$signature    = hash_hmac('sha256', $stringToSign, $clientSecret);

// Langkah 5: Sertakan di headers
```

```
$headers = [
    "X-CLIENT-KEY: {$clientKey}",
    "X-TIMESTAMP: {$timestamp}",
    "X-SIGNATURE: {$signature}",
    "Accept: application/json",
];
```

2.3 Fitur Keamanan Tambahan

FITUR	IMPLEMENTASI	TUJUAN
Anti Replay Attack	Timestamp window ±5 menit (konfigurasi)	Mencegah penggunaan kembali request yang sudah lama
Anti Timing Attack	Constant-time comparison (<code>hash_equals()</code>)	Mencegah analisis waktu respons untuk menebak signature
Client Key Validation	<code>hash_equals()</code> untuk membandingkan client key	Memastikan hanya klien terdaftar yang bisa mengakses
Audit Trail	Logging setiap request (sukses & gagal) ke <code>laravel.log</code>	Memudahkan investigasi dan pemantauan keamanan

⚠ Penting: Sinkronisasi Waktu Server

Pastikan waktu server pengirim request sudah disinkronkan dengan NTP server. Perbedaan waktu lebih dari 5 menit akan menyebabkan semua request ditolak dengan kode error **4010002** (Timestamp Expired). Gunakan `ntpdate pool.ntp.org` untuk sinkronisasi.

2.4 Konfigurasi Environment Server

.ENV — LARAVEL SERVER CONFIGURATION

```
# Konfigurasi VA Inquiry API Authentication
VA_INQUIRY_CLIENT_KEY=PPDBALHASRA
VA_INQUIRY_CLIENT_SECRET=AEkVExg92F
VA_INQUIRY_TIMESTAMP_TOLERANCE=5    # toleransi dalam menit (default: 5)

# Nilai ini dibaca di config/services.php:
# 'va_inquiry' => [
#   'client_key'      => env('VA_INQUIRY_CLIENT_KEY'),
#   'client_secret'   => env('VA_INQUIRY_CLIENT_SECRET'),
#   'timestamp_tolerance' => env('VA_INQUIRY_TIMESTAMP_TOLERANCE', 5),
# ],
```

3. Rate Limiting & Environment

3.1 Rate Limiting

Seluruh endpoint yang dilindungi HMAC dibatasi dengan middleware `throttle:60,1` , yaitu **maksimum 60 request per menit per IP address**. Jika terlampaui, server akan merespons dengan HTTP **429 Too Many Requests**.

ENVIRONMENT	BASE URL	KETERANGAN
🔧 Local Dev	http://localhost:8000	Jalankan dengan <code>php artisan serve</code> atau <code>composer dev</code>
🔧 Staging	https://staging.ppdb.alhasra.sch.id	Environment pengujian sebelum production
🔧 Production	https://ppdb.alhasra.sch.id	Environment live — wajib HTTPS

🔧 **Wajib HTTPS di Production**

Gunakan HTTPS di production untuk melindungi CLIENT_KEY, CLIENT_SECRET, dan SIGNATURE dari serangan man-in-the-middle. Jangan pernah mengirim kredensial melalui koneksi HTTP biasa.

4. Endpoint: VA Inquiry (GET by Path)

Mengambil data Virtual Account berdasarkan nomor VA di path URL. Cocok untuk integrasi REST murni.

Endpoint ini membaca nomor VA langsung dari URL path parameter. Nomor VA adalah string numerik hingga 30 karakter. Server akan mencari data payment terbaru dengan status apapun yang cocok dengan nomor VA tersebut.

Path Parameter

PARAMETER	TIPE	STATUS	KETERANGAN	CONTOH
va_number	string	WAJIB	Nomor Virtual Account (maks 30 karakter)	1234567890

Headers Wajib

HEADER	CONTOH NILAI	KETERANGAN
X-CLIENT-KEY	PPDBALHASRA	Client identifier
X-TIMESTAMP	2026-08-16T10:30:00+07:00	Timestamp ISO 8601
X-SIGNATURE	a3f9d2c1...	HMAC-SHA256 hex lowercase
Accept	application/json	Format response

CONTOH REQUEST — CURL

```
curl -X GET \
  "http://localhost:8000/api/va/1234567890" \
  -H "X-CLIENT-KEY: PPDBALHASRA" \
  -H "X-TIMESTAMP: 2026-08-16T10:30:00+07:00" \
  -H "X-SIGNATURE: a3f9d2c1b8e74f6a9d2e1c4b7f8a3d5e6c9b2f1a" \
  -H "Accept: application/json"
```

RESPONSE SUKSES —

200 OK

```
{
  "status": true,
  "responseCode": "2002500",
  "responseMessage": "Successful",
  "message": "Data Virtual Account berhasil ditemukan.",
  "data": {
    "nama_siswa": "Ahmad Fauzi Rahman",
    "fee_type": "Formulir", // Formulir | Daftar Ulang
    "va": "1234567890",
    "nominal": 500000, // Dalam Rupiah (float)
    "status": "pending" // pending | success
  }
}
```

RESPONSE ERROR —

404 NOT FOUND

(VA TIDAK DITEMUKAN)

```
{
  "status": false,
  "responseCode": "4042512",
  "responseMessage": "Bill not found",
  "message": "Data Virtual Account tidak ditemukan.",
  "data": null
}
```

RESPONSE ERROR —

401 UNAUTHORIZED

(SIGNATURE SALAH)

```
{
  "status": false,
  "responseCode": "4010003",
  "responseMessage": "Unauthorized: X-SIGNATURE tidak valid. Pastikan string-to-sign dan secret key benar.",
  "data": null
}
```

5. Endpoint: VA Inquiry (GET/POST by Body)

Mengambil data VA melalui body JSON (POST) atau query string (GET). Mendukung 3 nama parameter untuk kompatibilitas.

Endpoint ini menerima nomor VA melalui request body (POST) atau query string (GET). Mendukung 3 nama parameter alternatif untuk kompatibilitas dengan berbagai sistem perbankan. Parameter pertama yang ditemukan (tidak kosong) akan digunakan.

i Prioritas Parameter

Server membaca parameter dalam urutan: **va** → **va_number** → **virtualAccountNumber**. Parameter pertama yang tidak kosong akan digunakan sebagai nomor VA.

Request Body (POST) atau Query String (GET)

PARAMETER	TIPE	STATUS	KETERANGAN
va	string	WAJIB*	Nomor VA (parameter utama)
va_number	string	Alternatif	Nama alternatif untuk kompatibilitas gateway lain
virtualAccountNumber	string	Alternatif	Format standar BI SNAP untuk bank

* Salah satu dari ketiga parameter di atas wajib disertakan.

CONTOH REQUEST — POST DENGAN JSON BODY (CURL)

```
curl -X POST \
  "http://localhost:8000/api/va/inquiry" \
  -H "X-CLIENT-KEY: PPDBALHASRA" \
  -H "X-TIMESTAMP: 2026-08-16T10:30:00+07:00" \
  -H "X-SIGNATURE: a3f9d2c1b8e74f6a9d2e1c4b7f8a3d5e6c9b2f1a" \
  -H "Accept: application/json" \
  -H "Content-Type: application/json" \
  -d '{"va": "1234567890"}'
```

CONTOH REQUEST — GET DENGAN QUERY STRING (CURL)

```
curl -X GET \
  "http://localhost:8000/api/va/inquiry?va=1234567890" \
  -H "X-CLIENT-KEY: PPDBALHASRA" \
  -H "X-TIMESTAMP: 2026-08-16T10:30:00+07:00" \
  -H "X-SIGNATURE: a3f9d2c1b8e74f6a9d2e1c4b7f8a3d5e6c9b2f1a" \
  -H "Accept: application/json"
```

CONTOH REQUEST — FORMAT BI SNAP (VIRTUALACCOUNTNUMBER)

```
curl -X POST \
  "http://localhost:8000/api/va/inquiry" \
  -H "X-CLIENT-KEY: PPDBALHASRA" \
  -H "X-TIMESTAMP: 2026-08-16T10:30:00+07:00" \
  -H "X-SIGNATURE: a3f9d2c1b8e74f6a9d2e1c4b7f8a3d5e6c9b2f1a" \
  -H "Content-Type: application/json" \
  -d '{"virtualAccountNumber": "1234567890"}'
```

RESPONSE SUKSES —

200 OK

```
{
  "status"      : true,
  "responseCode" : "2002500",
  "responseMessage" : "Successful",
  "message"      : "Data Virtual Account berhasil ditemukan.",
  "data"         : {
    "nama_siswa" : "Ahmad Fauzi Rahman",
    "fee_type"    : "Formulir",
    "va"          : "1234567890",
    "nominal"     : 500000,
    "status"      : "pending"
  }
}
```

RESPONSE ERROR —

400 BAD REQUEST

(PARAMETER TIDAK DIBERIKAN)

```
{
  "status"      : false,
  "responseCode" : "4002500",
```

```
"responseMessage" : "Bad Request: Nomor Virtual Account (va) tidak diberikan.",
"message"         : "Parameter va / va_number / virtualAccountNumber wajib diisi.",
"data"            : null
}
```

6. Endpoint: VA Payment Notification

POST /api/va/payment-notify HMAC-SHA256 Required

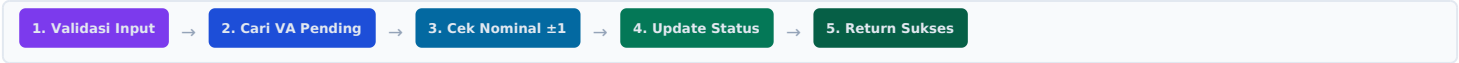
Menerima notifikasi pembayaran dari bank/gateway dan mengupdate status VA secara otomatis.

Endpoint ini menerima notifikasi pembayaran Virtual Account dari bank atau payment gateway. Server memvalidasi data, memverifikasi kesesuaian nominal (toleransi ±1 rupiah untuk pembulatan), lalu mengupdate status payment dan registration siswa secara otomatis.

6.1 Request Body (JSON) — Wajib: Content-Type: application/json

PARAMETER	TIPE	STATUS	VALIDASI	KETERANGAN
va_number	string	WAJIB	max:30 karakter	Nomor Virtual Account
amount	numeric	WAJIB	angka positif, min:1	Nominal yang dibayarkan (Rupiah)
paid_at	datetime	WAJIB	format: date/datetime	Waktu pembayaran (ISO 8601 atau Y-m-d H:i:s)
va_ref	string	Opsional	max:100 karakter	Nomor referensi transaksi dari bank
notes	string	Opsional	max:500 karakter	Catatan tambahan dari bank

6.2 Logika Pemrosesan



- Validasi input — semua field wajib harus ada dan valid
- Mencari record payment dengan `va_number` yang cocok dan status `pending`
- Jika sudah pernah dibayar (status `success`), kembalikan respons idempoten 200 tanpa mengubah data
- Validasi kesesuaian nominal dengan toleransi ±1 rupiah untuk menangani pembulatan
- Update status payment ke `success` dan isi `verified_at`, `va_ref`, `admin_note`
- Jika biaya ini adalah `sort_order == 1` (Formulir), update juga `payment_status` di tabel registration

Toleransi Nominal ±1 Rupiah

Server memvalidasi: `abs(amount - expectedAmount) <= 1`. Toleransi ini mengakomodasi perbedaan pembulatan kecil antar sistem bank. Jika selisih lebih dari 1 rupiah, request akan ditolak dengan kode **4002501**.

CONTOH REQUEST — CURL (LENGKAP)

```
curl -X POST \
  "http://localhost:8000/api/va/payment-notify" \
  -H "X-CLIENT-KEY: PPDBALHASRA" \
  -H "X-TIMESTAMP: 2026-08-16T10:30:00+07:00" \
  -H "X-SIGNATURE: a3f9d2c1b8e74f6a9d2e1c4b7f8a3d5e6c9b2f1a" \
  -H "Accept: application/json" \
  -H "Content-Type: application/json" \
  -d '{
    "va_number": "1234567890",
    "amount": 500000,
    "paid_at": "2026-08-16T10:25:00+07:00",
    "va_ref": "TXN-BTN-20260816-001",
    "notes": "Pembayaran Formulir PPDB 2026"
  }'
```

RESPONSE — 200 OK PEMBAYARAN BERHASIL DIPROSES

```
{
  "status": true,
  "responseCode": "2002500",
  "responseMessage": "Successful",
  "message": "Pembayaran berhasil diproses.",
  "data": {
    "payment_id": 42,
    "va_number": "1234567890",
    "nama_siswa": "Ahmad Fauzi Rahman",
    "fee_type": "Formulir",
    "amount": 500000, // Tagihan awal
    "paid_amount": 500000, // Yang dibayarkan
  }
}
```

```
"paid_at"      : "2026-08-16T10:25:00+07:00",
"status"       : "success"
}
}
```

RESPONSE — **200 OK** SUDAH DIBAYAR SEBELUMNYA (IDEMPOTEN)

```
{
  "status"      : true,
  "responseCode" : "2002501",
  "responseMessage" : "Payment Already Processed",
  "message"      : "Pembayaran untuk VA ini sudah diproses sebelumnya.",
  "data"         : null
}
```

RESPONSE — **422** NOMINAL TIDAK SESUAI

```
{
  "status"      : false,
  "responseCode" : "4002501",
  "responseMessage" : "Bad Request: Nominal tidak sesuai.",
  "message"      : "Nominal yang dibayarkan (Rp 450.000) tidak sesuai dengan tagihan (Rp 500.000).",
  "data"         : null
}
```

RESPONSE — **422** VALIDASI INPUT GAGAL

```
{
  "status"      : false,
  "responseCode" : "4002500",
  "responseMessage" : "Bad Request: Data tidak valid.",
  "errors"       : {
    "va_number" : ["Nomor Virtual Account (va_number) wajib diisi."],
    "amount"    : ["Nominal pembayaran (amount) wajib diisi."],
    "paid_at"   : ["Waktu pembayaran (paid_at) wajib diisi."]
  },
  "data"        : null
}
```

7. Endpoint: BTN Legacy Callback

POST

/api/btn/callback

Public — Tanpa Auth

Webhook yang menerima notifikasi pembayaran langsung dari sistem Legacy Bank BTN. Tidak memerlukan HMAC.

Endpoint ini dipanggil langsung oleh sistem backend Bank BTN saat terjadi pembayaran. Karena endpoint ini bersifat publik (tidak ada autentikasi HMAC), keamanan dilakukan di level jaringan (firewall / IP whitelist server BTN).

⚠ Keamanan Jaringan untuk Endpoint Publik

Karena tidak ada autentikasi HMAC, pastikan hanya IP server BTN yang diizinkan mengakses endpoint ini melalui konfigurasi firewall, Nginx `allow` directive, atau Laravel Trusted Proxies.

Request Body (JSON atau Form)

PARAMETER	TIPE	STATUS	KETERANGAN
va	string	WAJIB	Nomor Virtual Account
ref	string	Opsional	Nomor referensi transaksi dari BTN

Format Response (Format Khusus BTN Legacy)

FIELD	TIPE	KETERANGAN
rsp	string	"000" = sukses, "001" = gagal
rspdesc	string	Deskripsi hasil transaksi

CONTOH REQUEST — SIMULASI BTN CALLBACK (CURL)

```
curl -X POST \
  "http://localhost:8000/api/btn/callback" \
  -H "Content-Type: application/json" \
  -d '{"va": "1234567890", "ref": "BTN-REF-20260816"}'
```

RESPONSE — RSP:000 TRANSAKSI SUKSES

```
{ "rsp": "000", "rspdesc": "Transaction Success" }

// Jika sudah pernah diproses:
{ "rsp": "000", "rspdesc": "Transaction Already Processed" }
```

RESPONSE — RSP:001 GAGAL

```
{ "rsp": "001", "rspdesc": "Payment Record Not Found" }

// Kemungkinan nilai rspdesc lainnya:
// "Transaction Failed" — Payload kosong atau tidak valid
// "VA Number Missing" — Field 'va' tidak ada
```

8. Kode Respons & Error Handling

8.1 Tabel Kode Respons Lengkap

RESPONSECODE	HTTP STATUS	KETERANGAN	ENDPOINT
2002500	200	Sukses — Data ditemukan atau pembayaran diproses	Inquiry, Payment Notify
2002501	200	Pembayaran sudah diproses sebelumnya (idempoten)	Payment Notify
4000001	400	Format X-TIMESTAMP tidak valid (bukan ISO 8601)	Auth Middleware
4002500	400/422	Parameter wajib tidak diberikan atau validasi gagal	Inquiry, Payment Notify
4002501	422	Nominal yang dibayarkan tidak sesuai tagihan (selisih >1 rupiah)	Payment Notify
4010000	401	Header X-CLIENT-KEY, X-TIMESTAMP, atau X-SIGNATURE tidak ada	Auth Middleware

RESPONSECODE	HTTP STATUS	KETERANGAN	ENDPOINT
4010001	401	X-CLIENT-KEY tidak valid atau tidak dikenali sistem	Auth Middleware
4010002	401	X-TIMESTAMP kadaluarsa — selisih lebih dari 5 menit	Auth Middleware
4010003	401	X-SIGNATURE tidak valid — tidak cocok dengan yang diharapkan	Auth Middleware
4042512	404	Nomor VA tidak ditemukan di database	Inquiry, Payment Notify
429	429	Rate limit terlampaui — maks 60 request/menit per IP	Semua endpoint HMAC

8.2 Format Respons Error Standar

```
{
  "status"      : false,      // selalu false untuk semua error
  "responseCode" : "4010003", // kode error spesifik
  "responseMessage" : "Unauthorized: X-SIGNATURE tidak valid. Pastikan string-to-sign dan secret key benar.",
  "data"        : null       // selalu null untuk error
}
```

9. Contoh Implementasi Lengkap

9.1 PHP (cURL) — Inquiry & Payment Notify

PHP — VA INQUIRY (GET BY PATH)

```
<?php

function vaInquiry(string $vaNumber): array {
    $baseUrl      = 'http://localhost:8000';
    $clientId     = 'PPDBALHASRA';
    $clientSecret = 'AEkvExg92F';

    // 1. Generate timestamp dan signature
    $timestamp = (new DateTime())->format(DateTime::ATOM);
    $signature = hash_hmac('sha256', $clientId.'|'.$timestamp, $clientSecret);

    // 2. Kirim request
    $ch = curl_init();
    curl_setopt_array($ch, [
        CURLOPT_URL           => $baseUrl."/api/va/{$vaNumber}",
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_TIMEOUT        => 15,
        CURLOPT_HTTPHEADER    => [
            "X-CLIENT-KEY: {$clientId}",
            "X-TIMESTAMP: {$timestamp}",
            "X-SIGNATURE: {$signature}",
            "Accept: application/json",
        ],
    ]);
    $body = curl_exec($ch);
    $httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
    curl_close($ch);

    return ['http_code' => $httpCode, 'body' => json_decode($body, true)];
}

// --- Penggunaan ---
$result = vaInquiry('1234567890');
if ($result['http_code'] === 200 && $result['body']['status']) {
    $data = $result['body']['data'];
    echo "Nama      : " . $data['nama_siswa'] . PHP_EOL;
    echo "Biaya : Rp " . number_format($data['nominal'], 0, ',', '.') . PHP_EOL;
    echo "Status: " . $data['status'] . PHP_EOL;
} else {
    echo "Error: " . $result['body']['responseMessage'];
}
```

PHP — VA PAYMENT NOTIFICATION

```
function sendPaymentNotify(string $vaNumber, float $amount, string $paidAt, string $vaRef = ''): array {
    $baseUrl      = 'http://localhost:8000';
    $clientId     = 'PPDBALHASRA';
    $clientSecret = 'AEkvExg92F';

    $timestamp = (new DateTime())->format(DateTime::ATOM);
    $signature = hash_hmac('sha256', $clientId.'|'.$timestamp, $clientSecret);

    $payload = json_encode([
        'va_number' => $vaNumber,
        'amount'    => $amount,
        'paid_at'   => $paidAt,
        'va_ref'    => $vaRef,
        'notes'     => 'Notifikasi otomatis dari sistem',
    ]);

    $ch = curl_init();
    curl_setopt_array($ch, [
        CURLOPT_URL           => $baseUrl . '/api/va/payment-notify',
        CURLOPT_POST          => true,
        CURLOPT_POSTFIELDS    => $payload,
        CURLOPT_RETURNTRANSFER => true,
```

```
CURLOPT_HTTPHEADER => [
    "X-CLIENT-KEY: {$clientKey}",
    "X-TIMESTAMP: {$timestamp}",
    "X-SIGNATURE: {$signature}",
    "Content-Type: application/json",
    "Accept: application/json",
],
]);
$body      = curl_exec($ch);
$httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);
return ['http_code' => $httpCode, 'body' => json_decode($body, true)];
}

// --- Penggunaan ---
$r = sendPaymentNotify('1234567890', 500000, '2026-08-16T10:25:00+07:00', 'TXN-001');
echo $r['body']['message']; // "Pembayaran berhasil diproses."
```


9. Contoh Implementasi (lanjutan)

9.2 JavaScript (Node.js) — Inquiry & Payment Notify

JAVASCRIPT — VA INQUIRY & PAYMENT NOTIFY (NODE.JS V18+, BUILT-IN CRYPTO)

```
const crypto = require('crypto');

const BASE_URL      = 'http://localhost:8000';
const CLIENT_KEY     = 'PPDBALHASRA';
const CLIENT_SECRET = 'AEkvExg92F';

/** Buat headers HMAC-SHA256 untuk setiap request */
function makeHeaders(extraHeaders = {}) {
  const timestamp = new Date().toISOString();
  const stringToSign = `${CLIENT_KEY}|${timestamp}`;
  const signature = crypto
    .createHmac('sha256', CLIENT_SECRET)
    .update(stringToSign)
    .digest('hex');
  return {
    'X-CLIENT-KEY' : CLIENT_KEY,
    'X-TIMESTAMP'  : timestamp,
    'X-SIGNATURE'  : signature,
    'Accept'       : 'application/json',
    ...extraHeaders,
  };
}

/** VA Inquiry by path parameter */
async function vaInquiry(vaNumber) {
  const response = await fetch(`${BASE_URL}/api/va/${vaNumber}`, {
    headers: makeHeaders(),
  });
  const data = await response.json();
  if (data.status) {
    console.log(`Nama : ${data.data.nama_siswa}`);
    console.log(`Biaya : Rp ${data.data.nominal.toLocaleString('id-ID')}`);
    console.log(`Status: ${data.data.status}`);
  } else {
    console.error(`Error: ${data.responseMessage}`);
  }
  return data;
}

/** VA Payment Notification */
async function sendPaymentNotify(vaNumber, amount, paidAt, vaRef = null) {
  const payload = { va_number: vaNumber, amount, paid_at: paidAt };
  if (vaRef) payload.va_ref = vaRef;

  const response = await fetch(`${BASE_URL}/api/va/payment-notify`, {
    method : 'POST',
    headers : makeHeaders({ 'Content-Type': 'application/json' }),
    body    : JSON.stringify(payload),
  });
  const data = await response.json();
  console.log(`[${data.responseCode}] ${data.message}`);
  return data;
}

// ---- Contoh penggunaan ----
(async () => {
  // Inquiry VA
  await vaInquiry('1234567890');

  // Kirim notifikasi pembayaran
  await sendPaymentNotify(
    '1234567890',           // va_number
    500000,                // amount (Rupiah)
    '2026-08-16T10:25:00+07:00', // paid_at
    'TXN-BTN-20260816-001' // va_ref (opsional)
  );
});
```

```
})();
```

9.3 Python (requests) — Inquiry & Payment Notify

PYTHON 3 — PIP INSTALL REQUESTS

```
import hmac, hashlib, requests
from datetime import datetime, timezone, timedelta

BASE_URL      = 'http://localhost:8000'
CLIENT_KEY    = 'PPDBALHASRA'
CLIENT_SECRET = 'AEkvExg92F'

def make_headers() -> dict:
    """Generate auth headers dengan HMAC-SHA256 signature."""
    tz = timezone(timedelta(hours=7))
    ts = datetime.now(tz).isoformat(timespec='seconds')
    sig = hmac.new(
        CLIENT_SECRET.encode('utf-8'),
        f"{CLIENT_KEY}|{ts}".encode('utf-8'),
        hashlib.sha256
    ).hexdigest()
    return {
        'X-CLIENT-KEY' : CLIENT_KEY,
        'X-TIMESTAMP'  : ts,
        'X-SIGNATURE'   : sig,
        'Accept'       : 'application/json',
        'Content-Type'  : 'application/json',
    }

def va_inquiry(va_number: str) -> dict:
    """Inquiry data Virtual Account by path parameter."""
    r = requests.get(
        f"{BASE_URL}/api/va/{va_number}",
        headers=make_headers(),
        timeout=15
    )
    r.raise_for_status()
    result = r.json()
    if result['status']:
        d = result['data']
        print(f>Nama : {d['nama_siswa']}")
        print(f"Biaya : Rp {d['nominal']:,.0f}")
        print(f>Status: {d['status']}")
    return result

def payment_notify(va_number: str, amount: float, paid_at: str, va_ref: str = None) -> dict:
    """Kirim notifikasi pembayaran VA ke server PPDB."""
    payload = {'va_number': va_number, 'amount': amount, 'paid_at': paid_at}
    if va_ref:
        payload['va_ref'] = va_ref
    r = requests.post(
        f"{BASE_URL}/api/va/payment-notify",
        json = payload,
        headers = make_headers(),
        timeout = 15
    )
    r.raise_for_status()
    result = r.json()
    print(f"[{result['responseCode']}] {result['message']}")
    return result

# ---- Contoh penggunaan ----
if __name__ == '__main__':
    # 1. Cek data VA
    va_inquiry('1234567890')

    # 2. Kirim notifikasi pembayaran
    payment_notify(
        va_number = '1234567890',
        amount    = 500000,
        paid_at   = '2026-08-16T10:25:00+07:00',
        va_ref    = 'TXN-BTN-20260816-001'
    )
```


10. Integrasi Postman

10.1 Collection Variables & Pre-request Script

File koleksi Postman tersedia di root proyek: `PPDB-VA-API.postman_collection.json`. Import file ini ke Postman untuk langsung mencoba semua endpoint. Signature digenerate otomatis oleh Pre-request Script setiap kali request dikirim.

Collection Variables

VARIABLE	DEFAULT VALUE	KETERANGAN
BASE_URL	http://localhost:8000	Base URL server — ganti sesuai environment
CLIENT_KEY	PPDBALHASRA	Client key untuk autentikasi HMAC
CLIENT_SECRET	AekvExg92F	Secret key untuk generate signature
VA_NUMBER	1234567890	Nomor VA untuk keperluan testing

PRE-REQUEST SCRIPT (POSTMAN JAVASCRIPT)

```
// Script ini berjalan otomatis sebelum setiap request
const clientKey    = pm.collectionVariables.get("CLIENT_KEY");
const clientSecret = pm.collectionVariables.get("CLIENT_SECRET");
const moment       = require('moment');

// Generate timestamp ISO 8601
const timestamp = moment().format(); // "2026-08-16T10:30:00+07:00"

// Generate HMAC-SHA256 signature (menggunakan CryptoJS yang sudah built-in di Postman)
const stringToSign = clientKey + "|" + timestamp;
const signature     = CryptoJS.HmacSHA256(stringToSign, clientSecret)
    .toString(CryptoJS.enc.Hex);

// Set ke variabel sementara (digunakan di headers)
pm.variables.set("X-CLIENT-KEY", clientKey);
pm.variables.set("X-TIMESTAMP",  timestamp);
pm.variables.set("X-SIGNATURE",  signature);
```

Cara Import dan Penggunaan

- 1
- Buka Postman → klik **Import** → pilih file `PPDB-VA-API.postman_collection.json`
- 2
- Klik ikon titik tiga pada koleksi → pilih **Edit** → tab **Variables**
- 3
- Update nilai `BASE_URL` sesuai environment Anda (localhost / staging / production)
- 4
- Update `VA_NUMBER` dengan nomor VA yang valid dari database PPDB
- 5
- Klik **Save** → jalankan request → signature otomatis digenerate!

✓ **Signature Digenerate Otomatis**

Anda tidak perlu menghitung signature secara manual saat menggunakan Postman. Pre-request Script secara otomatis mengisi header `X-CLIENT-KEY`, `X-TIMESTAMP`, dan `X-SIGNATURE` setiap kali request dikirim.

11. FAQ & Troubleshooting

11.1 Masalah Umum & Solusi

❏ **Error 4010002: X-TIMESTAMP sudah kadaluarsa**

Penyebab: Perbedaan waktu antara server pengirim dan server PPDB lebih dari 5 menit.

Solusi: Sinkronkan jam server dengan NTP: `ntpdate pool.ntp.org`. Pastikan timezone sudah benar (WIB = UTC+7). Toleransi default adalah ±5 menit.

❏ **Error 4010003: X-SIGNATURE tidak valid**

Penyebab: Salah membuat string-to-sign, salah secret key, atau encoding tidak tepat.

Solusi: Pastikan format string-to-sign adalah `"CLIENT_KEY|TIMESTAMP"` (dipisahkan pipe). Signature harus dalam format hex **lowercase**. Gunakan CLIENT_SECRET yang sama persis dengan yang ada di server `.env`.

❏ **Error 4002501: Nominal tidak sesuai**

Penyebab: Amount berbeda lebih dari Rp 1 dari tagihan di database.

Solusi: Lakukan VA Inquiry terlebih dahulu untuk mendapatkan nominal yang tepat, kemudian gunakan nilai tersebut di Payment Notification.

⚠ **Error 429: Rate Limit Terlampaui**

Penyebab: Lebih dari 60 request dalam 1 menit dari IP yang sama.

Solusi: Implementasikan retry logic dengan exponential backoff. Tunggu minimal 60 detik sebelum mencoba kembali.

📖 **Tips: Cara Melihat Log Audit**

Setiap request (berhasil maupun gagal) dicatat ke Laravel log. Cek di:

```
# Lihat log terbaru (real-time)
tail -f storage/logs/laravel.log | grep "VA Inquiry"

# Log sukses: "VA Inquiry Auth OK"
# Log gagal : "VA Inquiry Auth FAILED"
# Log payment: "VA Payment Notify: SUCCESS"

# Cek konfigurasi via Tinker:
php artisan tinker
config('services.va_inquiry');
```